New upper and lower bounds on covering codes $K_q(n, R)$ for alphabets of size $5 \leq q \leq 21$

Mark Marosi*

August 2026

Abstract

Let $K_q(n, R)$ denote the minimum cardinality of a q-ary code of length n with covering radius R. We improve the known bounds on $K_q(n, R)$ in 81 cases. On the upper-bound side we give 23 improved bounds for $5 \leq q \leq 15$, obtained by two complementary search methods: an engineered focused local search seeded with structural constructions, and a large-neighbourhood search driven by exact full-space coverage transforms that evaluates every candidate codeword position simultaneously. These are, to our knowledge, the first improvements to any upper bound on $K_q(n, R)$ with $q \geq 5$ since the 2011 revision of Kéri's tables; several bounds decrease by more than 20%, e.g. $K_6(8, 4) \leq 166$ (previously 216) and $K_8(10, 5) \leq 1884$ (previously 2461). On the lower-bound side we give 58 improved bounds for $6 \leq q \leq 21$, obtained from the semidefinite programming hierarchy of Gijswijt and Polak, whose published results cover $q \leq 5$, by combining an exact-arithmetic reimplementation of the reduced program with a multiprecision solution pipeline. Every new lower bound is certified by a rational dual solution that a standalone, exact checker verifies from scratch; no floating-point computation is part of the trusted base. The same certificates delimit the method: on a number of further cells we certify that the true optimum of the relaxation lies below the best known bound, so no improvement can be obtained there at this level of the hierarchy. One cell is improved from both sides: $441 \leq K_6(10, 4) \leq 2751$, previously 417–2952. All codes and certificates are provided in machine-readable form together with standalone verifiers.

1 Introduction

Let $\mathbb{Z}_q^n$ denote the Hamming space of q-ary words of length n. A code $C \subseteq \mathbb{Z}_q^n$ has *covering radius* R if every word of $\mathbb{Z}_q^n$ is within Hamming distance R of some codeword, and

$$K_q(n, R) \;=\; \min\{\, |C| : C \subseteq \mathbb{Z}_q^n \text{ has covering radius} \leq R \,\}.$$

The function $K_q(n, R)$ is a central quantity of covering-code theory [1]; the binary case $K_2(n, 1)$ includes the domination number of the hypercube, and $K_3(n, 1)$ is the classical football pool problem.

*Department of Measurement and Information Systems, Budapest University of Technology and Economics (BME), Budapest, Hungary. email: marosi@mit.bme.hu. This work was carried out in collaboration with the AI system Claude (Anthropic): the author conceived and directed the research programme, and Claude produced the software, the mathematical elaboration, the computations, and this manuscript. See the Acknowledgments for a precise division of contributions.

Upper bounds on $K_q(n, R)$ are established by explicit constructions and lower bounds by combinatorial or convex-relaxation arguments; the state of the art is recorded in the tables of Kéri [2], which subsume and extend those of [1] and were last revised in November 2011. Recent work has concentrated on lower bounds for small alphabets: Wu and Chen [4] improved binary lower bounds via congruence arguments, and Gijswijt and Polak [3] obtained many new lower bounds for $q \le 5$ from a semidefinite programming hierarchy. For $q \ge 5$ we are not aware of any published improvement on either side since 2011; a systematic search of the literature (arXiv, DBLP, OpenAlex, Lobstein's covering-code bibliography, and the publication lists of the authors active in the area) found none.

This paper improves both sides of the tables in that range: 23 upper bounds for $5 \le q \le 15$ (Table 1) and 58 lower bounds for $6 \le q \le 21$ (Table 2).

Two structural observations locate the improvements. On the upper-bound side, Kéri's source annotations show that for moderate q and $R \ge 3$ most tabulated bounds are inherited from general constructions — the direct sum

$$K_q(n_1 + n_2,\ R_1 + R_2) \ \le\ K_q(n_1, R_1) \cdot K_q(n_2, R_2), \tag{1}$$

the rule $K_q(n+1, R) \le q\, K_q(n, R)$, the monotonicity $K_q(n+1, R+1) \le K_q(n, R)$, and product rules over alphabet factorizations — rather than from search in the cell itself. Such bounds decrease the furthest under direct search (by up to 23%); bounds previously obtained by dedicated search in the 2000s also decrease, by 1–6%. On the lower-bound side, the size of the symmetry-reduced Gijswijt–Polak semidefinite program depends only on n and not on q, so the restriction of the published results to $q \le 5$ reflects numerical precision rather than model size; solving the same relaxation in multiprecision arithmetic and certifying the results exactly yields improvements throughout $6 \le q \le 21$, concentrated on the cells whose incumbent lower bounds are close to the sphere-covering bound.

Section 2 states the results. Sections 3 and 4 describe the two upper-bound search methods and their implementation, Section 5 the lower-bound pipeline, and Section 6 the verification protocol applied to every claimed bound.

2 Results

Tables 1 and 2 list the new bounds. We note the following.

1. The cell $K_6(10, 4)$ is improved from both sides, from 417–2952 to 441–2751.

2. None of the upper-bound searches is known to be exhausted. The highest-redundancy cells (e.g. $K_7(9, 4)$, $K_6(10, 4)$, $K_8(10, 5)$) improved at every computational budget applied to them; the bounds marked $^{\downarrow}$ in Table 1 are the values at the time of writing, not limits of the method.

3. The lower-bound improvements are concentrated where the incumbents are weakest relative to the sphere-covering bound. All 58 improved cells carry Kéri keys `y`, `x`, `s` or `a` (bounds within a few percent of sphere-covering); the entire family $K_q(8, 2)$, $6 \le q \le 21$, is improved, with increments up to +14436 at $q = 21$, and the largest single increment is $K_{10}(10, 2) \ge 2699348$, previously 2676660. No cell whose incumbent is due to Haas, Halupczok and Schlage-Puchta (key `m`) is improved; Section 5.4 quantifies this limitation, which we certify rather than merely observe.

cell	previous bounds	new UB	Δ	Δ%	key	previous UB via
$K_5(11,5)$	103–625	**602**	-23	3.7%	`d`	Bhandari–Durairajan 1996
$K_6(7,3)$	70–246	**227**	-19	7.7%	`e`	$K_q(n{+}1,R) \le q\,K_q(n,R)$
$K_6(8,3)$	246–1080	**1005**	-75	6.9%	`f`	direct sum
$K_6(8,4)$	46–216	**166**	-50	23.1%	`e`	$K_q(n{+}1,R) \le q\,K_q(n,R)$
$K_6(9,4)$	136–738	**660**	-78	10.6%	`f`	direct sum
$K_6(9,5)$	32–144	**119**	-25	17.4%	`j`	alphabet product rule
$K_6(10,4)$	**441**–2952	**2751**$^{\downarrow}$	-201	6.8%	`f`	direct sum
$K_6(10,5)$	83–615	**486**$^{\downarrow}$	-129	21.0%	`f`	direct sum
$K_7(8,4)$	76–343	**316**	-27	7.9%	`c`	monotonicity $K_q(n{+}1,R{+}1) \le K_q(n,R)$
$K_7(9,4)$	264–1843	**1475**$^{\downarrow}$	-368	20.0%	`f`	direct sum
$K_7(10,5)$	160–1225	**981**$^{\downarrow}$	-244	19.9%	`f`	direct sum
$K_8(6,4)$	15–20	**19**$^{\downarrow}$	-1	5.0%	`n`	Kéri–Östergård 2005
$K_8(8,4)$	118–512	**505**	-7	1.4%	`c`	monotonicity $K_q(n{+}1,R{+}1) \le K_q(n,R)$
$K_8(8,5)$	29–90	**85**$^{\downarrow}$	-5	5.6%	`z`	Kéri 2009
$K_8(10,5)$	287–2461	**1884**$^{\downarrow}$	-577	23.4%	`f`	direct sum
$K_8(10,6)$	58–342	**341**$^{\downarrow}$	-1	0.3%	`n`	Kéri–Östergård 2005
$K_9(9,5)$	120–729	**684**$^{\downarrow}$	-45	6.2%	`c`	monotonicity $K_q(n{+}1,R{+}1) \le K_q(n,R)$
$K_9(10,5)$	481–3969	**3393**$^{\downarrow}$	-576	14.5%	`f`	direct sum
$K_{10}(9,5)$	174–1088	**1055**$^{\downarrow}$	-33	3.0%	`f`	direct sum
$K_{12}(6,4)$	30–41	**39**$^{\downarrow}$	-2	4.9%	`q`	Rivas Soriano 2005–2008
$K_{13}(6,4)$	35–46	**45**$^{\downarrow}$	-1	2.2%	`q`	Rivas Soriano 2005–2008
$K_{14}(6,4)$	40–52	**50**$^{\downarrow}$	-2	3.8%	`q`	Rivas Soriano 2005–2008
$K_{15}(6,4)$	46–59	**57**$^{\downarrow}$	-2	3.4%	`c`	monotonicity $K_q(n{+}1,R{+}1) \le K_q(n,R)$

Table 1: New upper bounds. "Previous bounds" are the lower–upper pairs of Kéri's tables [2]; a bold lower bound is from Table 2 of this paper. "Key" is the source annotation of the previous upper bound in [2], expanded in the last column. A $^{\downarrow}$ marks bounds whose descent had not stalled when this version was prepared; all other bounds resisted a dedicated attack at $M-1$.

cell	sphere bound	previous LB (key)	new LB	SDP value$^{1/3}$	known UB
$K_6(8,2)$	2267	2276 (y)	**2367**	2366.9301	5184
$K_6(9,2)$	10653	10900 (y)	**10965**	10964.2020	29808
$K_6(9,3)$	881	921 (y)	**926**	925.3962	4752
$K_6(10,3)$	3739	3815 (y)	**3836**	3835.4257	19347
$K_6(10,4)$	411	417 (s)	**441**	440.7459	2952
$K_7(7,2)$	1031	1035 (x)	**1081**	1080.1873	2401
$K_7(8,2)$	5454	5457 (x)	**5631**	5630.3303	15129
$K_7(8,3)$	439	457 (y)	**471**	470.4031	2337
$K_7(9,2)$	29870	29889 (s)	**30562**	30561.8700	94587
$K_7(9,3)$	2070	2077 (y)	**2143**	2142.3820	8575
$K_7(10,2)$	168041	168042 (x)	**170632**	170631.3884	420175
$K_8(8,2)$	11741	11766 (y)	**12033**	12032.2538	29920
$K_8(8,3)$	813	829 (y)	**856**	855.2348	4096
$K_8(9,4)$	403	409 (s)	**429**	428.3487	2944
$K_9(8,2)$	23181	23184 (x)	**23642**	23641.8903	59049
$K_9(8,3)$	1411	1413 (x)	**1464**	1463.3308	6561
$K_9(9,3)$	8537	8544 (y)	**8685**	8684.2332	29889
$K_9(9,4)$	690	703 (s)	**722**	721.2544	5103
$K_9(10,3)$	54142	54144 (x)	**54600**	54599.9308	177147
$K_{10}(7,2)$	5666	5676 (y)	**5824**	5823.5368	13500
$K_{10}(8,2)$	42717	42772 (y)	**43423**	43422.6390	123904
$K_{10}(9,2)$	333556	333560 (x)	**337394**	337393.0428	1000000
$K_{10}(9,4)$	1123	1130 (s)	**1160**	1159.6365	8500
$K_{10}(10,2)$	2676660	2676660 (a)	**2699348**	2699347.1750	7040000
$K_{10}(10,4)$	6808	6886 (y)	**6915**	6914.8596	45900
$K_{11}(7,2)$	8977	9106 (y)	**9193**	9192.1126	14641
$K_{11}(8,2)$	74405	74415 (s)	**75448**	75447.9273	161051
$K_{12}(8,2)$	123665	123772 (y)	**125156**	125155.9824	323040
$K_{12}(8,3)$	5512	5577 (y)	**5605**	5604.5932	26920
$K_{13}(6,2)$	2162	2169 (x)	**2233**	2232.7347	6127
$K_{13}(7,2)$	20183	20189 (x)	**20545**	20544.8593	47111
$K_{13}(8,2)$	197562	197563 (x)	**199633**	199632.7737	371293
$K_{13}(8,3)$	8085	8086 (x)	**8193**	8192.9316	28561
$K_{14}(6,2)$	2881	2955 (y)	**2964**	2963.6350	7203
$K_{14}(7,2)$	28952	29273 (y)	**29404**	29403.7678	78934
$K_{14}(8,2)$	305105	305294 (y)	**307909**	307908.9214	671728
$K_{15}(6,2)$	3766	3812 (y)	**3862**	3861.5469	10625
$K_{15}(8,2)$	457578	457584 (x)	**461294**	461293.0985	1166533
$K_{16}(6,2)$	4841	4848 (x)	**4951**	4950.2767	10752
$K_{16}(7,2)$	55566	55600 (y)	**56237**	56236.1518	172032
$K_{16}(8,2)$	668894	669207 (y)	**673723**	673722.1052	1954905
$K_{17}(7,2)$	74757	75429 (y)	**75559**	75558.5708	252735
$K_{17}(8,2)$	955977	955978 (x)	**962145**	962144.9827	3038049
$K_{17}(8,3)$	29475	29478 (x)	**29652**	29651.0059	172557
$K_{17}(8,4)$	1446	1458 (y)	**1464**	1463.8773	9801
$K_{18}(6,2)$	7664	7741 (y)	**7804**	7803.2298	15309
$K_{18}(8,2)$	1339162	1339650 (y)	**1346931**	1346930.6400	3779136
$K_{19}(6,2)$	9468	9479 (x)	**9624**	9623.2345	20890
$K_{19}(7,2)$	128968	128972 (x)	**130081**	130080.7695	396910
$K_{19}(7,3)$	4236	4237 (x)	**4282**	4281.5827	18737
$K_{19}(8,2)$	1842635	1842639 (x)	**1852296**	1852295.0865	5564881
$K_{20}(7,2)$	165911	167165 (y)	**167204**	167203.3213	563200
$K_{20}(7,3)$	5166	5174 (x)	**5215**	5214.7205	21202
$K_{20}(8,2)$	2494884	2495614 (y)	**2506759**	2506758.0751	7929856
$K_{21}(6,2)$	14012	14131 (y)	**14201**	14200.9736	37481
$K_{21}(7,3)$	6243	6249 (x)	**6294**	6293.4996	28812
$K_{21}(8,2)$	3329184	3329193 (x)	**3343629**	3343628.7531	9529569
$K_{21}(8,3)$	82338	82341 (x)	**82595**	82594.8774	453789

Table 2: New lower bounds, each certified by an exact rational dual solution (Section 5). "Sphere bound" is $\lceil q^n/|B_R|\rceil$; "previous LB" is the value of [2] with its source key: y = improved sphere-covering bound, x = Chen–Honkala 1990, s = Lang–Quistorff–Schneider 2006–2007, a

Remark 1. *An audit of the tables of [2] against their own derivation rules found one internal gap: the tabulated* $K_{17}(7,2) \le 252735$ *is weaker than the bound* $17 \cdot K_{17}(6,2) = 245208$ *implied by the tabulated* $K_{17}(6,2) \le 14424$ *and the rule* $K_q(n{+}1,R) \le q\,K_q(n,R)$*. We record this as an observation; it involves no new construction.*

3 Upper bounds I: engineered local search

3.1 The search

For fixed (q,n,R,M) we search for a code of size M minimizing the number of uncovered words. The state is a multiset of M codewords together with the coverage count $\mathrm{cnt}(w)$ of every $w \in \mathbb{Z}_q^n$; a move changes one coordinate of one codeword.

Lemma 1. *Let* c' *be obtained from* c *by setting coordinate* p *to* $v \ne c_p$*. Then the set of words leaving the radius-*R *ball of* c *and the set of words entering the ball of* c' *are both indexed by the words at distance exactly* R *from* c *in the coordinates other than* p*; consequently a move updates exactly* $2S$ *coverage counters with* $S = \binom{n-1}{R}(q-1)^R$*, the two sets are disjoint, and distinct counter updates address distinct words.*

Proof. Since $d(w,c') = d(w,c) + [\,w_p = c_p\,] - [\,w_p = v\,]$, a word can leave the ball of c only if $d(w,c) = R$ and $w_p = c_p$, and can enter the ball of c' only if $d(w,c') = R$ and $w_p = v$; writing such a word as its restriction to the coordinates $\ne p$ (at distance exactly R from the restriction of c) plus its value at p gives the stated bijections, disjointness, and injectivity. □

The search is a focused local search in the WalkSAT lineage, close in spirit to the tabu searches that produced much of the original tables [6, 5]: select a random uncovered word u; take as candidates every codeword at distance exactly $R+1$ from u moved one coordinate towards u (if no codeword is that close, the codewords at minimum distance from u); evaluate all candidates exactly; commit a best candidate, ties broken uniformly at random; on stagnation, reassign one codeword to an uncovered word. In parameter studies on $K_6(6,3)$, exact evaluation of all candidates outperformed sampling 24 or 64 of them (final uncovered counts 0 against 82 and 18), while tabu tenures and WalkSAT-type noise degraded performance, and simulated annealing lost to the focused search despite a 33× higher move rate.

Each cell is seeded with a code rebuilt from the construction that produced the incumbent bound (for $K_6(10,4)$, for instance, the direct sum (1) of optimal $K_6(4,1)$ and $K_6(6,3)$ codes, both found by direct search and verified). Descent proceeds by remove-and-repair: delete the codeword whose private coverage is smallest and repair the deficit by search; on success, recurse at $M-1$.

3.2 Implementation

Profiling shows that candidate evaluation accounts for 92.9%, 98.5% and 99.6% of the run time on three reference cells of increasing size ($K_6(6,3)$ at $M=41$, $K_6(8,4)$ at $M=169$, $K_8(9,4)$ at $M=2944$), and that the cost of one evaluated sphere pattern rises from 1.9 ns to 7.4 ns as the counter array grows from cache-resident (93 kB) to DRAM-resident (268 MB). Three exact reformulations of the evaluation reduce both the patterns walked and the bytes touched.

variant	$K_6(6,3)$ 20 000 it	$K_6(8,4)$ 600 it	$K_8(9,4)$ 50 it
reference implementation	1.00×	1.00×	1.00×
peeled walk, hoisted offsets	1.35×	1.29×	1.13×
+ 8-bit state array	1.28×	1.65×	1.65×
+ 2-bit state array	0.82×	1.34×	2.44×
shared sphere, 8-bit state	1.32×	1.95×	1.95×
+ uncovered-word list	2.21×	5.57×	6.06×
with 2-bit state	2.14×	**6.32**×	**9.86**×

Table 3: Speedup in CPU time at a fixed iteration budget, median of 9 runs (6 on the largest cell). Every row performs the identical search: final uncovered counts agree exactly, seed by seed, across rows. Interquartile ranges are 0.9%, 2.2% and 5.6% of the median.

Lemma 2. *With $D(w) = \{j : w_j \neq c_j\}$, a word w leaves the ball of c under the move at coordinate p if and only if $|D(w)| = R$ and $p \notin D(w)$; it enters the ball of the moved codeword if and only if $|D(w)| = R+1$, $p \in D(w)$ and $w_p = v$.*

(i) Shared sphere. By Lemma 2, with $T = \#\{w : |D(w)| = R,\ \mathrm{cnt}(w) = 1\}$ and $H_j = \#\{w : |D(w)| = R,\ \mathrm{cnt}(w) = 1,\ j \in D(w)\}$, the number of words newly uncovered by the move at p equals $T - H_p$ for every p simultaneously. One enumeration of the full distance-R sphere ($\binom{n}{R}(q-1)^R$ patterns) replaces $R+1$ enumerations of $\binom{n-1}{R}(q-1)^R$ patterns each.

(ii) Explicit uncovered set. The entering side of Lemma 2 concerns only words with $\mathrm{cnt}(w) = 0$, and membership of the entering set is decided in $O(n)$ from the digits of the word. Maintaining the uncovered set U explicitly replaces $(R+1)\binom{n-1}{R}(q-1)^R$ counter reads by $O(n|U|)$ work, profitable whenever $n|U| < (R+1)\binom{n-1}{R}(q-1)^R$; on the reference cells this inequality holds by two to four orders of magnitude throughout the useful life of a run, and the solver falls back to the sphere walk when it fails. Together, (i) and (ii) reduce the counter reads per candidate from $2(R+1)\binom{n-1}{R}(q-1)^R$ to $\binom{n}{R}(q-1)^R$ (4.0–5.6× on the reference cells).

(iii) Two-bit read path. Evaluation only asks whether a counter is 0 or 1; a two-bit side array $\min(\mathrm{cnt}(w), 2)$ serves the read path while exact counters remain for commits. On $K_8(9,4)$ this shrinks the read working set from 268 MB to 33.5 MB, inside the last-level cache.

Since $T - H_p$ is independent of the written value v, and the entering counts from U are indexed by (p, v) at no extra cost, the same enumeration yields the exact objective change of all $n(q-1)$ single-coordinate moves of the codeword; widening the candidate neighbourhood in this way is beneficial on loose cells (see below).

Every reformulation computes the same objective differences as the reference implementation and therefore follows the same search trajectory from the same seed. This was verified bit-for-bit over 24 configurations covering $2 \le q \le 7$, $R = 0$, $R = n-1$, and sizes above and below the optimum; in addition, the uncovered count reported by every run is required to equal the count that the standalone verifier computes from the written code file.

Tables 3 and 4 quantify the effect. All figures are process CPU time (the host was shared, so wall-clock is not meaningful); the first experiment fixes the iteration budget, so identical trajectories make it a controlled comparison, and the second measures CPU time to a fixed target uncovered count over 20 seeds.

Negative results. Several natural optimizations fail for structural reasons, which we record.

cell	M	target	identical search	wide neighbourhood
$K_6(6,3)$	41	$\lvert U\rvert \le 30$	$2.03\times$ $[1.99, 2.13]$	$0.28\times$
$K_6(8,4)$	169	$\lvert U\rvert \le 20$	$6.12\times$ $[6.06, 6.62]$	$8.03\times$ $[5.75, 11.02]$
$K_8(9,4)$	2944	$\lvert U\rvert \le 10\,800$	$8.07\times$ $[6.19, 10.72]$	$11.04\times$ $[8.06, 12.17]$

Table 4: Median per-seed speedup in CPU time to target over 20 seeds, with percentile-bootstrap 95% confidence intervals. In the fourth column the optimized solver reached the target at the same iteration as the reference on all seeds, so the ratio is a pure implementation speedup; the fifth column also widens the candidate neighbourhood, which changes the search (fewer iterations to target on the two loose cells, a $3.6\times$ loss on the tight one).

Caching move evaluations between iterations requires evaluation spheres to be disjoint, i.e. $d(c, c') > 2R+2$; since $2R+2 \ge n$ on every cell of interest, every commit invalidates every cache entry. Maintaining the singly-covered analogue of U pays only if $n|U_1| < \binom{n}{R}(q-1)^R$, which fails by one to two orders of magnitude — uncovered words are rare by construction, singly covered words are not. Translation symmetry reduction (fixing one codeword at 0^n) yields no measurable benefit, since a local search does not revisit states. A deterministic focus rule (always the longest-uncovered word) is severely harmful, destroying the randomization that lets focused search leave plateaus. Restart schedules help only where the run-time distribution to the target is heavier than exponential, which occurs on tight cells (observed speedup $4.2\times$, $p = 0.002$, on $K_6(6,3)$ at a hard target) and not on the loose cells this paper improves (coefficient of variation 0.20–0.47; $p = 1.0$).

3.3 Structured codes

Free search over subsets of $\mathbb{Z}_q^n$ is the wrong representation for cells whose optima are algebraic. Two restricted solvers complement it.

Unions of cosets of a linear code. When q is a prime power and $M = c\,q^k$, we search codes of the form $C = \bigcup_{i=1}^{c}(x_i + L)$ with L a linear $[n,k]_q$ code. Covering then depends only on syndromes: C covers $\mathbb{Z}_q^n$ if and only if the c syndromes of the x_i cover the q^{n-k}-element syndrome space under the weight distribution of coset leaders, reducing a q^n-word problem to a q^{n-k}-syndrome one. Cells whose incumbents free search could not even reproduce are settled by this solver in under a second, e.g. $K_8(8,4) \le 512 = 8^3$ and $K_5(11,5) \le 625 = 5^4$; the descents to the new records 505 and 602 then start from verified incumbents.

Translation quotients. A second solver searches codes invariant under $x \mapsto x + \mathbf{1}$ with a free remainder, working in the quotient of size q^{n-1}. On $K_6(8,4)$, neither a fully invariant code (which provably stalls: a single free codeword cannot repair a deficient orbit) nor a fully free search solves the cell, whereas a mostly invariant code with a free remainder solves it reliably; at equal core-seconds on the largest cells the quotient search outperforms free search by 17%.

3.4 Solver selection and portfolio

The production configuration was chosen by benchmarking four independently developed implementations — the engineered free search of Section 3.2, a low-level SIMD variant, the structured solvers of Section 3.3, and a portfolio scheduler — under a fixed protocol: identical interface and resource limits, public development instances, held-out evaluation instances, 15 seeds per

instance, and scoring by independently verified output only (a full verified cover scores 1000; a partial cover scores the negated uncovered fraction; invalid output scores below any valid partial). No component won uniformly: the structured solvers dominated the held-out evaluation, the engineered evaluator was fastest on large cells, the SIMD variant on cache-resident cells, and many single-threaded chains beat multi-threaded ones except on the largest cells. The production engine merges the four accordingly: an algebraic first phase (coset solver over all feasible k; seeds from recorded codes and direct sums over all splits), then a portfolio of independent chains whose composition is set by measured thresholds on q^n and the redundancy $q^n/(M|B_R|)$; the incumbent is written atomically on every improvement; a finishing pass re-reads the final code from disk, removes duplicates, and recomputes coverage by direct ball marking. Under the frozen protocol the merged engine performed at least as well as every component on every instance (14/20 held-out solves against the best component's 12/20).

All record searches of Table 1 with $q^n < 10^8$ were produced by this engine on the 64 CPU cores of a single NVIDIA GH200 node, with per-cell effort between seconds and a few core-days.

4 Upper bounds II: search by exact full-space transforms

Cells with $q^n \gtrsim 10^8$ resist the method of Section 3: each chain's working set exceeds the CPU caches and the per-move sphere walk becomes memory-latency-bound. Two direct GPU ports of the local search were implemented and measured first: a formulation of candidate evaluation as integer matrix products (to engage tensor cores) is starved by its contracted dimension of only $nq \approx 50$ and never approached CPU throughput; and a port running 256 independent chains in one persistent kernel reaches $0.09\times$, $0.10\times$ and $1.24\times$ the throughput of the full 64-core socket on cells with $q^n = 4.7 \cdot 10^4$, $1.7 \cdot 10^6$ and $1.3 \cdot 10^8$ respectively — at best one extra socket, obtained only where the CPU is already at its worst. These measurements motivated a change of algorithm rather than of implementation.

4.1 The coverage transforms

For $S \subseteq \mathbb{Z}_q^n$ and $0 \le d \le R$ let $N_d^S(x) = \#\{u \in S : d_H(x,u) = d\}$. All $R+1$ arrays N_d^S can be computed simultaneously for *every* $x \in \mathbb{Z}_q^n$ by a sequential transform over the coordinate axes.

Proposition 1. *Initialize A_0 as the indicator of S and $A_1 = \cdots = A_R = 0$. For each axis $j = 1, \dots, n$, update along every one-dimensional fiber $\{x + te_j\}_{t \in \mathbb{Z}_q}$:*

$$A_d[a] \;\leftarrow\; A_d[a] + \Big(\textstyle\sum_b A_{d-1}[b]\Big) - A_{d-1}[a], \qquad d = R, \dots, 1.$$

After all n axes, $A_d = N_d^S$ for every d. The total work is $\Theta(nR\,q^n)$ additions.

Proof. Induction on the number of processed axes: after processing axes $1, \dots, j$, $A_d(x)$ counts the $u \in S$ that differ from x in exactly d of the first j coordinates and agree with x elsewhere. The update adds, for each fiber position a, the count of elements at distance $d-1$ in the first $j-1$ axes whose j-th coordinate differs from a. □

The update is branchless and uniform over q^{n-1} independent fibers — a bandwidth-bound workload on which a GPU operates at its design point. On the GH200 used here, the full transform takes 63 ms at $q^n = 7^{10}$ and 143 ms at 8^{10} with 32-bit counters (memory footprint $4(R+1)+2$ bytes per word of the space).

Two instances of Proposition 1 give global exact information that pointwise local search cannot afford. With S the set of uncovered words, $g(x) = \sum_{d \le R} N_d^S(x)$ is the exact number of words newly covered by a codeword placed at x, for every candidate position x simultaneously; with S the set of words covered exactly once, $\ell(c) = \sum_{d \le R} N_d^S(c)$ is the exact number of words that would be uncovered by deleting c, for every codeword c.

4.2 The search

The engine is a large-neighbourhood search over these primitives: starting from a seed code (a direct-sum construction or a recorded code), delete a set of codewords chosen by low ℓ-value (mixed with randomized removals), then rebuild by repeatedly placing a codeword at the maximizer of the current gain map g, recomputing or locally correcting g after each placement; a completed cover at size M triggers a descent attempt at $M-1$ by the same remove-and-rebuild step. Each placement is thus globally optimal with respect to exact coverage information, in contrast to the single-coordinate moves of Section 3; the engine performs far fewer moves of far higher quality. The greedy rebuild is also an effective constructor from an empty code: on $K_8(10,4)$ ($q^n = 1.07 \cdot 10^9$) it builds a full cover of size 14946 in three minutes, a task on which the CPU engine failed outright.

Five of the bounds of Table 1 were produced by this engine on cells with $2.8 \cdot 10^8 \le q^n \le 3.5 \cdot 10^9$, within hours and including reductions of 19.9% ($K_7(10,5)$), 23.4% ($K_8(10,5)$) and 14.5% ($K_9(10,5)$); all were verified by the independent protocol of Section 6. The remaining GPU-accessible cells of the tables ($q^n \le 4 \cdot 10^9$ with 32-bit counters resident in GPU memory; larger spaces via host-coherent memory) were still under attack at the time of writing.

5 Lower bounds: exact certificates from a multiprecision SDP pipeline

5.1 The relaxation and its exact reconstruction

Gijswijt and Polak [3] bound $K_q(n,R)^3$ from below by a semidefinite program over variables indexed by orbits of triples of words under $\mathrm{Aut}(q,n) = S_q \wr S_n$, with positive-semidefiniteness constraints from the block-diagonalization of the Terwilliger algebra of the Hamming scheme, linear constraints from sphere-covering inequalities, and matrix-cut constraints (their Theorem 4.18). The size of the reduced program depends only on n — at $n = 11$ it has 339 variables and 126 blocks of order at most 12 — while q enters only through the coefficients; published computations stop at $q \le 5$.

We reimplemented the reduced program with the entire model generation in exact integer arithmetic and validated the implementation four ways: (a) a self-test that substitutes explicit verified covering codes into the model and checks every constraint in exact rational arithmetic, with objective exactly $|C|^3$; (b) a coefficient-level comparison against the authors' published code, matching every generated coefficient over six (q,n,R) cells spanning $3 \le q \le 7$; (c) reproduction of the published values: of 49 published table values recomputed, 44 agree to display precision, and 18 of the 22 published improved lower bounds attempted are reproduced exactly at the integer level; (d) the invariant that no certified lower bound may exceed a known upper bound, checked against all 1145 table cells and never violated.

5.2 Certification

Numerical solutions are used only as suggestions. By weak duality, any feasible point of the dual program — multipliers $y_k \geq 0$ for the linear constraints and positive semidefinite matrices Y_b for the blocks — certifies the bound it attains. Each numerical dual solution is rounded to rationals over a dyadic denominator, each Y_b is displaced strictly into the semidefinite cone, and, should any dual inequality fail in exact arithmetic, the entire dual is scaled by a dyadic factor $\theta \leq 1$, which restores feasibility at proportional cost in the bound.

The resulting certificate is validated by a standalone checker (about 700 lines, Python standard library only) that re-derives the entire program from (q, n, R) in exact integer arithmetic, verifies $y_k \geq 0$, verifies each $Y_b \succeq 0$ by exact LDL^{T} factorization over the rationals, verifies every dual inequality in integer arithmetic, and extracts the integer bound by exact cube-root comparison. The checker also verifies that the covering inequality named in the certificate is itself valid, so that an unsound constraint cannot be smuggled in. The bounds of Table 2 rest on this checker alone.

5.3 Numerical solution

Two numerical issues govern whether a good dual point can be found at all. First, conditioning: the optimum $|C|^3$ may exceed 10^{13} while every constraint constant is $O(1)$, and the objective coefficients span ten orders of magnitude. All computations therefore use a geometric-mean equilibration — normalize the objective by the cube of the sphere-covering bound and give each variable the magnitude profile of the square root of its objective contribution — applied entirely in powers of two, so that the map back to the unscaled dual is exact. On $K_6(8, 3)$ this raises the certified value from 183 to 240 against a true optimum of 239.52.

Second, precision. In IEEE double precision, interior-point solvers fail beyond optima of roughly $1.5 \cdot 10^9$ (cube root $\approx$1150), which excludes most of the cells with the largest headroom. We therefore solve in multiprecision arithmetic with SDPA-GMP [8] at 200-bit precision, compiled natively for the target architecture. The problem files are written with exact dyadic coefficients, the multiprecision dual is parsed into exact rationals, and rounding is performed *in the equilibrated coordinates* over denominator 2^{128}, with the displacement into the semidefinite cone tied to the rounding grain. With these choices, every certificate underlying Table 2 attains $\theta = 1$: nothing of the numerically obtained bound is lost in certification. As an end-to-end anchor, the pipeline reproduces the published 512-bit value $K_5(8, 2) \geq 861$ of [3] exactly.

5.4 Extent and limits of the method

The improvements land where the analysis of the hierarchy predicts. At this level the semidefinite bound exceeds the sphere-covering bound by a factor that decreases in q (for $(n, R) = (8, 4)$: 1.43 at $q = 4$ down to 1.15 at $q = 7$), so it overtakes the tabulated bounds precisely where those are close to sphere-covering: the improved cells all carry keys `y`, `x`, `s` or `a`, and complete families such as $(n, R) = (8, 2)$, $6 \leq q \leq 21$, fall at once.

The certificates also establish limits. On more than twenty further cells — among them $K_q(7, 3)$ for $15 \leq q \leq 18$, $K_q(8, 3)$ for $q \in \{14, 15, 16, 20\}$, and $K_q(8, 4)$ for $18 \leq q \leq 21$ — the pipeline certifies, with $\theta = 1$, a value strictly *below* the tabulated bound; since the certified value is the true optimum of the relaxation to the precision used, no improvement is possible on those cells at this level of the hierarchy. In the extreme case $R = 1$ the relaxation adds nothing at $q \geq 6$: for $K_6(7, 1)$ the certified optimum equals the sphere-covering bound 7776

exactly. Against the strong combinatorial bounds of Haas, Halupczok and Schlage-Puchta (key `m`, 13–52% above sphere-covering) the relaxation is weaker throughout the range considered. We also note that the fractional covering *linear* program cannot improve any of these cells: since $\mathrm{Aut}(q, n)$ acts transitively on $\mathbb{Z}_q^n$, an averaging argument shows its optimum is exactly $q^n/|B_R|$, the sphere-covering bound.

6 Verification

Every bound in this paper was verified independently of the software that produced it.

Upper bounds. Every claimed code, re-read from its published file, was checked by four methods, two of which share no code with the other two: (i) ball marking over a q^n bitmap; (ii) meet-in-the-middle bitset covering over a coordinate split; (iii) a min-distance scan over all q^n words, which also reports the exact covering radius; (iv) R-fold neighbourhood dilation of the code's indicator array, which computes no distances at all. A fifth, independently written dilation verifier re-confirmed all 23 codes from the published files immediately before this version was prepared; the largest check covers $9^{10} = 3.49 \cdot 10^9$ words. The distinct-codeword count of every file is checked against the claimed M.

Lower bounds. Every certificate was re-validated by the exact checker of Section 5.2 immediately before this version was prepared (196 certificates: 58 improving, 28 matching the tabulated value, none exceeding a known upper bound).

The ancillary files contain all codes (one codeword per line, digits `0`–`9` then `a`–`z` for $q > 10$), all 58 record certificates, and both standalone verifiers (`verify_cov.py` and `certify.py`), each dependency-free.

7 Concluding remarks

The upper-bound experiments show a consistent pattern: bounds inherited from general constructions (slack 4–10× over the sphere-covering bound) fall under direct search, by up to 23%; bounds set by dedicated search in the 2000s fall by a few percent; bounds that were already the object of intensive search or algebraic construction (slack $\leq 3\times$) resist. The transform-based engine of Section 4 extends the reachable range to $q^n \approx 4 \cdot 10^9$ and none of the large-cell descents has yet stalled, so further improvements are expected from computation alone. On the lower-bound side, the multiprecision pipeline has exhausted neither the alphabet range nor the hierarchy: the van Wee-type strengthenings of the linear constraints, used in [3] for $q = 2$ only, and the conditional use of known upper bounds are both untried at $q \geq 6$, while the certified negative results delimit what the present level can achieve.

Since the tables of [2] have had no maintainer since 2011, a merged machine-readable bounds table (Kéri 2011, the post-2011 literature, and the present improvements) is published alongside the code.

Data availability

The codes, the certificates, both verifiers, and a README are ancillary files of the arXiv version of this paper. The full search, certification, and verification source code, together with the merged bounds table, is available at `https://github.com/Mapika/coldcase`.

Acknowledgments

This work is AI-assisted in the strong sense: the search software, the verification protocol, the constructions, the certificates, and this manuscript were designed, implemented, and operated autonomously by Claude (Anthropic), which should be regarded as the primary contributor to the results. The author's contributions were the conception and the continued direction of the research programme: the initiating idea of attacking record tables frozen since the 2000s with modern large-scale search; the choice to pursue certified lower bounds alongside constructive upper bounds; the direction to reformulate the search in dense, branchless form suited to accelerator hardware, which led to the transform-based engine of Section 4; the proposal of the competitive solver-selection methodology of Section 3.4; and the computational resources.

References

[1] G. Cohen, I. Honkala, S. Litsyn, A. Lobstein, *Covering Codes*, North-Holland, Amsterdam, 1997.

[2] G. Kéri, Tables for bounds on covering codes, `https://old.sztaki.hu/~keri/codes/` (last updated November 2011).

[3] D. Gijswijt, S. Polak, Semidefinite lower bounds for covering codes, arXiv:2504.01932, 2025.

[4] Y. Wu, C. Chen, Improved lower bounds on the domination number of hypercubes and binary codes with covering radius one, Discrete Mathematics 347 (2024) 113752.

[5] P. J. M. van Laarhoven, E. H. L. Aarts, J. H. van Lint, L. T. Wille, New upper bounds for the football pool problem for 6, 7 and 8 matches, J. Combin. Theory Ser. A 52 (1989) 304–312.

[6] P. R. J. Östergård, Constructing covering codes by tabu search, J. Combin. Des. 5 (1997) 71–80.

[7] G. Kéri, P. R. J. Östergård, Bounds for covering codes over large alphabets, Des. Codes Cryptogr. 37 (2005) 45–60.

[8] M. Nakata, A numerical evaluation of highly accurate multiple-precision arithmetic version of semidefinite programming solver: SDPA-GMP, -QD and -DD, Proc. IEEE Int. Symp. on Computer-Aided Control System Design, 2010, 29–34.

[9] A. Florath, A Lean 4 formalization of q-ary covering codes with a proof-carrying database of certified bounds, arXiv:2606.09600, 2026.

A The code proving $K_6(8,4) \leq 166$

The largest relative single improvement among the converged cells ($216 \to 166$) is printed for self-containedness; each row lists codewords of $\mathbb{Z}_6^8$, and the machine-readable version is the ancillary file `K6_8_4_M166.txt`.

01430421 12541532 23052043 30103154
45214205 50325310 03405225 14510330
25021441 30132552 41243003 52354114
03134001 14245112 25350223 30401334
51512445 52010550 02442243 13553304
24004415 35115520 40220031 51331142
02303231 13414342 24525453 35030504
40141015 51252120 05143544 10254055
21305100 32410211 43521322 54032433
04301535 13432040 20543151 31054202
42105313 53210434 04043240 15154351
20205402 31310513 42421024 53532135
05111133 10222244 21333355 32444400
43555511 54000022 01045452 12150503
23201014 34312125 45423230 50534341
00544125 11055230 22100341 33211452
44322503 55433014 02231500 13342011
24453122 35504233 40015344 51120455
05223315 10334420 21445531 32550042
43001153 54112204 03411050 14542101
25033212 30144323 41255434 52300445
04450305 15501410 20012521 31123032
42234143 53345254 00052413 11103524
22214035 33324140 44430251 55541302
02115012 13220123 24331234 35442345
40553450 51004501 01024314 12135425
23240530 34351041 45402152 50513203
05352332 10413443 21514554 52025505
43130110 54241221 04205542 15315053
20420104 31531215 42042320 53153431
01213111 12324222 23435333 34540444
45051555 42131403 00300252 11411303
22522414 33033525 44104030 55255141
05535024 10040135 21151240 32202351
44313402 54424513 01500043 12051134
23122205 34233310 45344421 50455532
03314404 10402515 25530020 31021031
41132242 52243353 12005001 03402520
31455113 23344502 55203541 32515205
30325000 02023150 14021545 54142054
34124521 03523252 45250342 00522424
44553234 53044110